

CS112: Wednesday 7 October, 12:15 to 1:20

Monday we turned `PyList` into `PyList<Item>`. The `typedef` went, every method moved into the header, `PyList.cpp` was deleted, and `PyList<string>` held three names in a test that had only ever held `int`.

Today: the one in the standard library. `vector` does everything `PyList` does, it has been there the whole time, and the algorithms that go with it do not take a container at all.

TIME	BEAT
12:15	Daily quiz
12:20	1. <code>vector</code> , which you have already written
12:34	2. <code>begin()</code> , <code>end()</code> , and one past the last
12:50	3. <code>find</code> and <code>sort</code> over a pair of iterators
13:06	4. What indexing costs
13:16	5. Close

Beat 1. `vector`, which you have already written (14 min)

Open `w05-vectors/main.cpp`. Four names pushed back, one assertion on `stds[3]`, one on `size()`. It runs.

```
vector<string> stds;  
stds.push_back("Micaiah");  
assert(stds[3] == "Diego");  
assert(stds.size() == 4);
```

Talk to your neighbor. Line by line, what does our `PyList` call each of these?

`push_back` is `append`. `size()` is `getSize()`. `operator[]` is the one we wrote twice. There is no `getCapacity()` in the assertions, and `capacity()` exists.

Two differences, on the board:

PYLIST**VECTOR**

<code>v[99]</code> out of range	throws <code>invalid_argument</code>	reads whatever is there
checked access	<code>getIndex</code>	<code>at()</code>

Ours is kinder than the real thing. That is not a compliment to ours.

Beat 2. `begin()`, `end()`, and one past the last (16 min)

```
stds.begin()    // the position of stds[0]
stds.end()      // the position AFTER stds[3]
```

An iterator is a position, not a value. `*it` is the value at it.

Talk to your neighbor. `end()` points at something nobody may read. Why have it at all?

Two answers, both on the board:

```
vector<string>::iterator it;
for (it = stds.begin(); it != stds.end(); ++it)
    cout << *it << " ";           // the range is [begin, end)

if (stds.begin() == stds.end())
    cout << "empty";              // no special case needed
```

A half-open range has one end you reach and one you stop at, and an empty container needs no test of its own.

Beat 3. `find` and `sort` over a pair of iterators (16 min)

```
#include <algorithm>

vector<string>::iterator it = find(stds.begin(), stds.end(),
    "Diego");

if (it == stds.end()) cout << "not here\n";
else cout << *it << " at " << (it - stds.begin());

sort(stds.begin(), stds.end());
```

Why does `find` take two iterators instead of the vector?

Because then it works on half a vector, on an array, and on a list nobody has written yet. The algorithm never learns what it is walking.

What does `find` return when it fails, and why that?

`end()`. It is the one position guaranteed to exist and to be no element, so failure needs no special value and no exception.

Run it. Then run it again with `"Kwesi"`.

Beat 4. What indexing costs (10 min)

```
stds[0]
stds[3]
stds[499999]
```

Talk to your neighbor. Which of those three takes longest?

None of them. One multiply and one add, from the same arithmetic as before: `base + sizeof(Item) * index`. The size of the vector never enters it.

$$\text{address}(v[i]) = \text{base} + \text{sizeof}(\text{Item}) \times i$$

That is what a contiguous block buys, and it is the reason `push_back` has to move everything when it grows: the block cannot be extended, only replaced.

Friday puts a number on that.

Beat 5. Close (4 min)

You can now read the standard library instead of only calling it. `vector` is the class you wrote, written by people who have argued about it for thirty years.

Friday: Prof. Victor Norman, on what `append` costs.